

Improving PAC-Bayes bounds for neural nets

Joshua Tan, Manoj Manikandan, Rohan Navaratnam

Department of Computer Science, University of Manchester

Introduction

Recent works like Dziugaite and Roy 2017 and Zhou et al. 2018 have shown that one can obtain non-vacuous “computational” PAC-Bayesian bounds on the risk of neural nets. In this work we improve on existing theoretical PAC-Bayesian risk bounds by using data-dependent priors sensitive to the geometrical properties of training and by considering carefully chosen clusters of multiple nets in tandem.

Notation

All losses considered are multi-class margin losses. Training data-sets (denoted as S) will be assumed to be of size m and contained in a ball of radius B , centered at the origin in $\mathbb{R}^n$. Define $f_A : \mathbb{R}^n \rightarrow \mathbb{R}^k$ to be a depth- d neural-network with maximum width h , whose ℓ^{th} layer has weight matrix A_ℓ . The first $d - 1$ layers of f_A use the RELU non-linear activation. A is the vector of parameters formed by concatenating vectorized layer matrices, i.e. A denotes the vector $[\text{vec}(A_1); \dots; \text{vec}(A_d)]$ whose each coordinate is a distinct trainable weight in the net.

Define $\beta(A) = \left(\prod_{\ell=1}^d \|A_\ell\|_2\right)^{1/d}$. We will omit the argument A whenever the neural-network under consideration is clear from the context. *Clearly β^d upperbounds the Lipschitz-constant for f_A .*

Let $\{A_i \in \mathbb{R}^{\dim(A)} \mid i = 1, \dots, k_1\}$ denote a set of $k_1 \geq 2$ neural net weight vectors on the underlying architecture of f_A . Let $\{p_i \mid i = 1, \dots, k_1\}$ be a set of non-zero scalars s.t $\sum_i p_i = 1$. Define $\beta_i := \beta(A_i)$.

Controlled output perturbation with a mixture of Gaussians sampling for the weights

Theorem (1)

Suppose the following condition holds for some $\varepsilon > 0$,

$$\forall i \in \{1 \dots k_1\}, \mathbf{x} \in S \quad \|f_{A_i}(\mathbf{x}) - f_A(\mathbf{x})\| \leq \varepsilon \|f_A(\mathbf{x})\|$$

Then for every $\gamma > \varepsilon \max_{\mathbf{x} \in S} \|f_A(\mathbf{x})\|$, we have,

$$\mathbb{P}_{A' \sim MG(\text{posterior})} \left[\max_{\mathbf{x} \in S} \|f_{A'}(\mathbf{x}) - f_A(\mathbf{x})\| > 2\gamma \right] \leq \delta$$

Where $MG(\text{posterior})(\mathbf{w}) = \sum_i p_i \mathcal{N}_{(A_i, \sigma^2)}(\mathbf{w})$

$$\text{and } \sigma \leq \frac{1}{\sqrt{2h \log\left(\frac{2d h k_1}{\delta}\right)}} \min_{1 \leq i \leq k_1} \min \left\{ \frac{\beta_i}{d}, \frac{\gamma}{k_1 \varepsilon d B p_i \beta_i^{d-1}} \right\}.$$

□

Now we will use this above noise resilience theorem about nets to get finer data-dependent PAC-Bayesian risk bounds on neural nets.

Defining “Nice” training dataset

We call a training dataset S as (ε, γ) –nice w.r.t neural weight vectors A and $\{A_i\}_{i=1}^{k_1}$ if it satisfies the following two conditions,

- $\max_{\mathbf{x} \in S} \|f_{A_i}(\mathbf{x}) - f_A(\mathbf{x})\| \leq \varepsilon \|f_A(\mathbf{x})\|, \forall 1 \leq i \leq k_1$
- $\gamma > \varepsilon \max_{\mathbf{x} \in S} \|f_A(\mathbf{x})\|$

Descent Lemma

The following lemma establishes the per-step bound for gradient descent on smooth functions. The original paper states the constrained case but for Nesterov’s accelerated gradient descent in the smooth function case, we only need the unconstrained version.

Lemma (1)

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ be a convex and β -smooth function. Let $x, y \in \mathbb{R}^n$ and $x^+ = x - \frac{1}{\beta} \nabla f(x)$. Then the following holds true:

$$f(x^+) - f(y) \leq \nabla f(x)^\top (x - y) - \frac{1}{2\beta} \|\nabla f(x)\|^2$$

Nesterov’s accelerated gradient descent

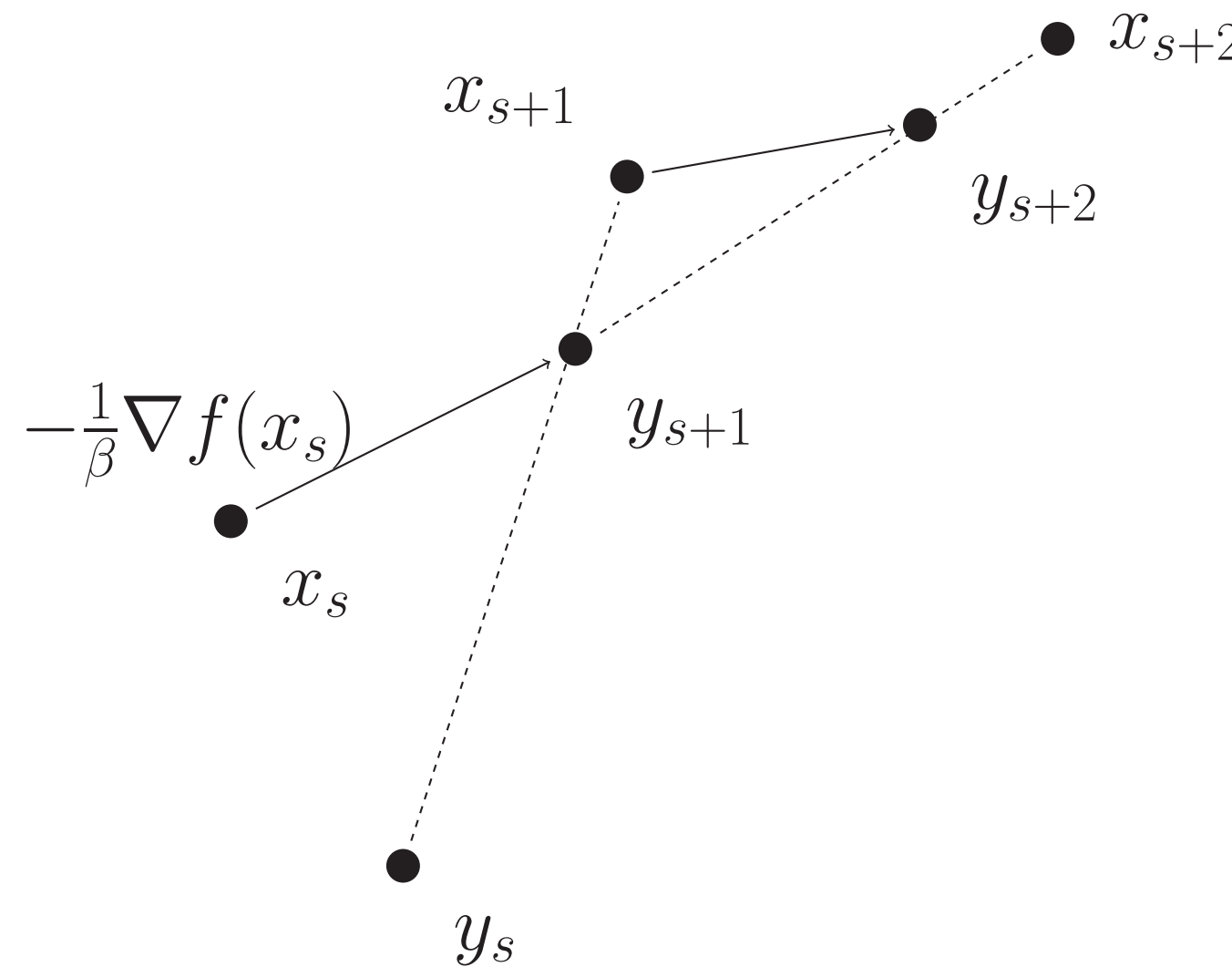


Figure: Illustration of Nesterov’s accelerated gradient descent.

First we introduce the following sequences:

$$\lambda_0 = 0, \quad \lambda_t = \frac{1 + \sqrt{1 + 4\lambda_{t-1}^2}}{2}, \quad \text{and } \gamma_t = \frac{1 - \lambda_t}{\lambda_{t+1}}$$

Nesterov’s accelerated gradient descent is described as follows. Start at an arbitrary initial point $x_1 = y_1$ and then iterate the following equations for $t \geq 1$

$$y_{t+1} = x_t - \frac{1}{\beta} \nabla f(x_t)$$
$$x_{t+1} = (1 - \gamma_t) y_{t+1} - \gamma_t y_t$$

Theorem (2)

Let f be a convex and β -smooth function, then Nesterov’s accelerated gradient descent satisfies

$$f(y_t) - f(x^*) \leq \frac{2\beta \|x_1 - x^*\|^2}{t^2}$$

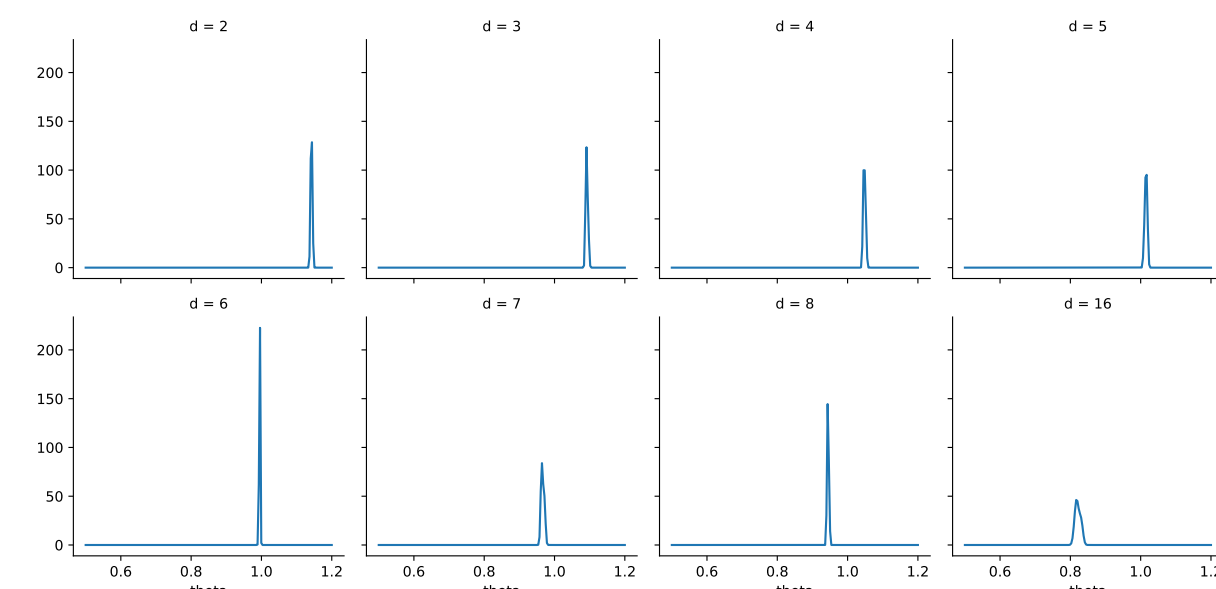
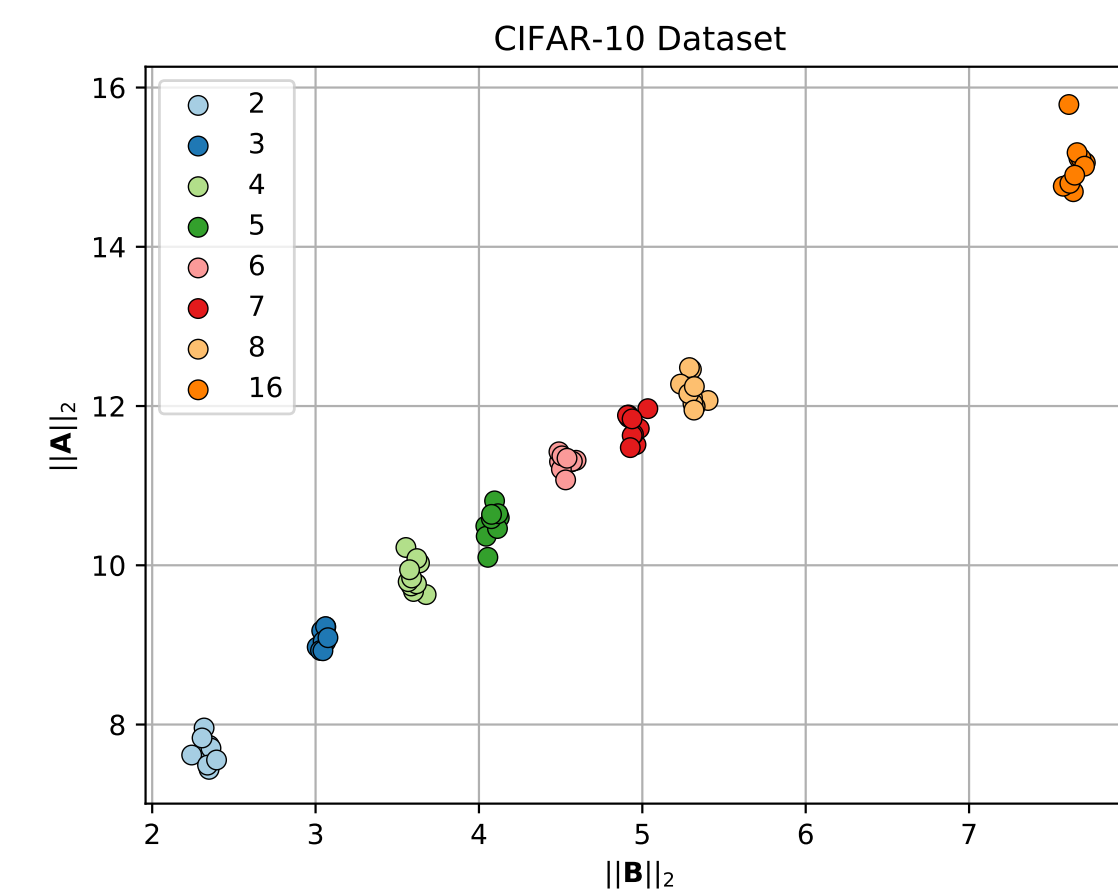
This gives us the $O(1/t^2)$ complexity. [Write more about it here](#)

Main Ideas for Theorem 2

- Nesterov’s accelerated gradient descent uses the ideas from classical momentum gradient descent (cite), the gradient is computed at x_t to produce y_{t+1} but the next x_{t+1} is not simply y_{t+1} . x_{t+1} overshoots y_{t+1} in the direction of the most recent step y_t with some “momentum” given by $(1 - \gamma_t)$ and γ on y_{t+1} and y_t respectively.
- The sequence λ_t comes from the need to satisfy $\lambda_{t-1}^2 = \lambda_t^2 - \lambda_t$.

Demonstration of stability of the relevant geometrical properties of training

In the *figure to the left* we plot the initial parameter norm $\|B\|_2$ vs the final parameter norm $\|A\|_2$ for training nets of different depths at width 100 on the CIFAR-10 dataset. Thus its demonstrated that the multiplicative factor with which the norm increases during training is fairly stable to architectural changes.



In the *figure to the right* is displayed the Gaussian kernel density estimate on 10 trials of the experiment (at different depths and 100 width) measuring the angular deviation of the weight vector under training. This deflection angle only decreases slightly with depth and showed negligible dependence over the widths tried.

References

-  Dziugaite, Gintare Karolina and Daniel M Roy (2017). “Computing nonvacuous generalization bounds for deep (stochastic) neural networks with many more parameters than training data”. In: *arXiv preprint arXiv:1703.11008*.
-  Zhou, Wenda et al. (2018). “Non-vacuous generalization bounds at the imagenet scale: a PAC-bayesian compression approach”. In: *arXiv preprint arXiv:1804.05862*.